

Task 10: Searching Algorithms & Logical Reasoning

Data Structures & Algorithms — Linear Search, Binary Search + Blood Relations (Reasoning) · Task 10

Task 8 aur 9 mein humne data ko **sort** karna seekha (Bubble, Selection, Insertion → Merge, Quick). Ab asli sawaal — sort kiya hi kyun? Kyunki **sorted data mein kuch bhi dhundna bahut fast ho jata hai**. Is task mein hum **Searching** seekhenge. Hardik ke liye Binary Search ke problems hain (logic simple, bas template pakka karna hai), aur Akshara ke liye DSA ke saath **Reasoning — Blood Relations** bhi add hua hai.

1 Theory: Searching Algorithms

Searching ka matlab: kisi collection (array/list) mein ek **target** element hai ya nahi, aur hai toh kis index par.

Linear Search $O(N)$

Shuru se end tak ek-ek element check karo jab tak target na mil jaye. Array sorted ho ya na ho — dono pe chalta hai. Simple hai, lekin bade data pe slow.

Analogy: Shaadi ke album mein ek photo dhundna jab photos random order mein lagi hon — pehle page se ek-ek karke palatna padega.

→ [GeeksforGeeks: Linear Search](#)

Binary Search $O(\log N)$

Sirf **sorted** array pe kaam karta hai. Beech wala element (`mid`) dekho — target usse chhota hai toh sirf **left half** mein dhundo, bada hai toh sirf **right half** mein. Har step mein search space **aadha** ho jata hai.

Analogy: Dictionary mein "Mango" dhundna. Tum page 1 se nahi padhte — beech se kholte ho; "P" dikha toh peeche jaate ho, "D" dikha toh aage. Ya phir "1 se 100 ke beech number guess karo" game — hamesha beech wala number bolo, max 7 guesses mein jawab mil jayega.

→ [GeeksforGeeks: Binary Search](#)

Kitna fast hai?

Array size (N)	Linear Search (worst case)	Binary Search (worst case)
100	100 checks	7 checks
10 lakh (10^6)	10,00,000 checks	20 checks
100 crore (10^9)	1,00,00,00,000 checks	30 checks

Binary Search Template (Java — Iterative)

```
static int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;    // overflow-safe
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;    // right half
        else high = mid - 1;                        // left half
    }
    return -1;    // not found
}
```

4 Golden Rules:

1. Array **sorted** hona chahiye — warna galat answer aayega (aur koi error bhi nahi dikhega!).
2. Loop condition `low <= high` — jab `low == high` ho tab bhi ek element check karna baaki hai.
3. `mid = low + (high - low) / 2` likho, `(low + high) / 2` nahi — bade numbers pe overflow ho sakta hai.
4. Update hamesha `low = mid + 1` / `high = mid - 1` — sirf `mid` likha toh infinite loop ka khatra.

Java mein built-in `Arrays.binarySearch(arr, key)` bhi hota hai — lekin element na mile toh kya return karta hai, woh Case 4 mein dekhenge.

→ [Java Docs: Arrays class](#)

2 DSA Practice Problems

Akshara ke liye

Basics → Searching — pichle task (First and Last Digit, Reverse Array) ka agla step

1. Sum of Digits CODECHEF

First and Last Digit ka agla step — ab sirf pehla-aakhri nahi, **har digit** nikal kar sum karna hai.

Hint: $n \% 10$ se last digit milta hai, $n / 10$ se woh digit hat jaata hai. Jab tak n zero na ho, repeat.

→ codechef.com/problems/FLOW006

2. Reverse The Number CODECHEF

Number ko ulta karo — jaise 1234 → 4321, aur 1200 → 21 (aage ke zero nahi aane chahiye).

*Hint: $rev = rev * 10 + digit$ — Reverse an Array wala idea, bas digits ke saath.*

→ codechef.com/problems/FLOW007

3. Array Search GFG

Array mein x ka **pehla** index return karo, na mile toh -1. Yahi Linear Search hai.

→ [GeeksforGeeks: Array Search](https://www.geeksforgeeks.org/array-search/)

4. Check if Array is Sorted GFG

Ek hi loop mein check karo ki har element apne pichle element se bada ya barabar hai. (Binary Search lagane se pehle array sorted hona zaroori hai — yahi check hai!)

→ [GeeksforGeeks: Check if Array is Sorted](https://www.geeksforgeeks.org/check-if-array-is-sorted/)

5. Find Numbers with Even Number of Digits LEETCODE 1295

Har number ke digits gino (Q1 wala loop) aur batao kitne numbers mein digits ki count even hai.

→ leetcode.com/problems/find-numbers-with-even-number-of-digits

6. Binary Search (Basic) GFG

Sorted array mein k hai ya nahi — true/false return karo. Theory wala template seedha use karo, aur ek baar paper pe dry run zaroor karna.

→ [GeeksforGeeks: Binary Search](https://www.geeksforgeeks.org/binary-search/)

Hardik ke liye

Binary Search Practice — logic simple hai, template pakka karna hai

1. Binary Search LEETCODE 704

Sorted array mein target ka index return karo, na mile toh -1. Pure template practice — bina dekhe likh ke dekho.

→ leetcode.com/problems/binary-search

2. Search Insert Position LEETCODE 35

Target mile toh uska index, na mile toh woh index jahan insert karne par array sorted rahe.

Hint: Loop khatam hone ke baad Low kahan khada hai? Wahi answer hai.

→ leetcode.com/problems/search-insert-position

3. First Bad Version LEETCODE 278

Versions 1 se n tak hain; ek version ke baad saare "bad" hain. Sabse pehla bad version dhundo, kam se kam API calls mein.

Hint: Bad mila toh answer save karo aur left jao — shayad usse pehle bhi koi bad ho. n bahut bada ho sakta hai, toh mid overflow-safe formula se hi nikalo.

→ leetcode.com/problems/first-bad-version

4. **Sqrt(x)** LEETCODE 69

`Math.sqrt()` ke bina x ka square root nikalo (decimal hata ke) — jaise $8 \rightarrow 2$.

*Hint: 0 se x tak ke numbers pe binary search. $mid * mid$ int mein overflow kar sakta hai — Long use karo.*

→ leetcode.com/problems/sqrtx

5. **Find First and Last Position of Element in Sorted Array** LEETCODE 34

Sorted array mein target pehli aur aakhri baar kahan aata hai — `[first, last]` return karo.

Hint: Bas do baar binary search. Match milne par ruko mat — first ke liye left jaate raho, last ke liye right. (Code Modification Challenge se connected hai!)

→ leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array

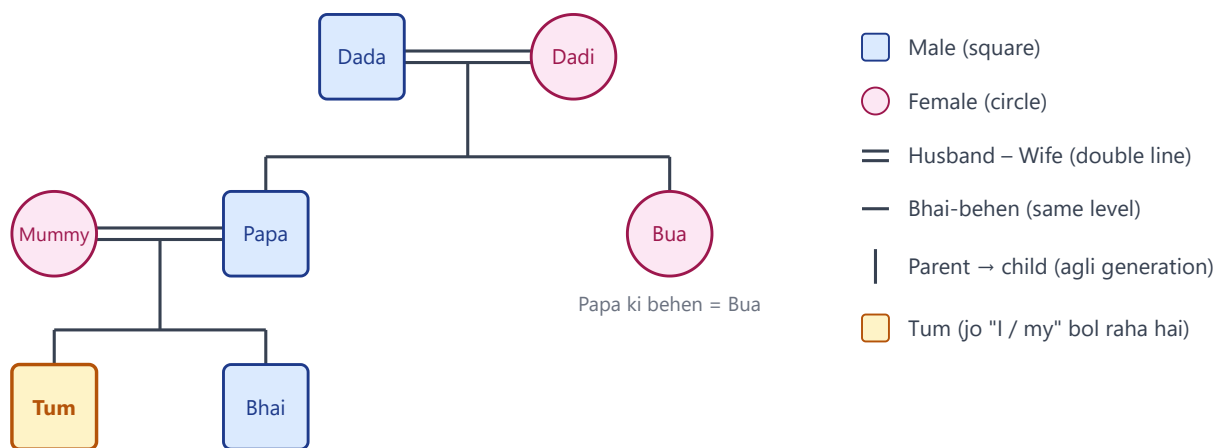
3 Reasoning: Blood Relations Akshara ke liye

Blood Relation kya hota hai?

Is type ke questions mein rishte seedhe nahi bataye jaate — ghuma-phira ke bataye jaate hain, jaise "*mere papa ke papa ki beti*". Tumhe usko khol ke final rishta batana hota hai (jawab: **Bua**). Yeh topic placements, SSC, bank exams aur CAT jaise almost har aptitude test mein aata hai — aur asal mein yeh bhi logic building hi hai, bilkul code ka dry run karne jaisa: ek-ek step, dhyaan se.

Sochne ka tarika — "Apni Family se Socho"

1. **Khud ko "I / my" maan lo.** Question mein jo bol raha hai, uski jagah khud ko rakho aur har rishta apni real family pe map karo — "my mother's brother" = *mere Mama*. Chehra yaad aa gaya, toh answer bhi aa gaya.
2. **Hindi mein socho, English mein likho.** English ka ek "uncle" = Chacha, Mama, Mausa, Fufa — sab! Hindi rishte precise hote hain, isliye confusion kam hota hai.
3. **"My" se shuru karo, ek-ek step aage badho.** Jaise: "the daughter of the mother of my father" → my father = Papa → Papa ki mother = Dadi → Dadi ki daughter = **Bua**. Beech ke har step ka naam likhte jao.
4. **"Only son / only daughter" trap.** Agar ladka bole "my mother's only son" — woh **khud** hai! Ladki bole "my mother's only daughter" — woh **khud** hai. Isliye pehle bolne wale ka gender dekho.
5. **Gender assume mat karo.** Sirf letter (A, B, C) se gender pakka nahi hota. Jab tak diya na ho, "sibling / child" socho.
6. **Mushkil lage toh family tree banao.** Neeche wale symbols se 30 second mein tree ban jaata hai — aur galti ke chances zero.



Example: Dadi ki beti aur Papa ki behen — dono raaste same insaan tak jaate hain: Bua.

Rishton ki Cheat Sheet (Hindi → English)

Hindi	Kaun?	English (exam mein yeh likhna hai)
Dada / Dadi	Papa ke papa / mummy	Paternal grandfather / grandmother
Nana / Nani	Mummy ke papa / mummy	Maternal grandfather / grandmother
Chacha / Tau	Papa ke chhote / bade bhai	Paternal uncle
Bua / Fufa	Papa ki behen / unke pati	Paternal aunt / uncle (by marriage)
Mama / Mami	Mummy ke bhai / unki wife	Maternal uncle / aunt
Mausi / Mause	Mummy ki behen / unke pati	Maternal aunt / uncle (by marriage)
Bhabhi / Jija	Bhai ki wife / behen ke pati	Sister-in-law / Brother-in-law
Saas / Sasur	Pati ya patni ki mummy / papa	Mother-in-law / Father-in-law
Bahu / Damad	Bete ki wife / beti ke pati	Daughter-in-law / Son-in-law
Pota / Poti	Bete ka beta / beti	Grandson / Granddaughter
Chachera / Mamera bhai-behen	Chacha / Mama ke bachhe	Cousin

15 Practice Questions

Har question ke liye BloodRelations.md mein **answer + steps (ya chhota family tree)** likhna hai — sirf answer nahi chalega.

LEVEL 1 — WARM-UP

Q1. Rohan (a boy) says, "Priya is the daughter of my mother's only son." How is Priya related to Rohan?

Q2. Pointing to a boy, Neha says, "He is the son of my Dada's only son." How is the boy related to Neha?

Q3. Introducing a girl, Amit says, "Her mother is the only daughter of my mother." How is Amit related to the girl?

Q4. Pointing to a boy, Anjali says, "His mother is the only daughter of my mother." How is Anjali related to the boy? (*Q3 se compare karo!*)

Q5. X is the brother of Y. Y is the sister of Z. Z is the father of W. How is X related to W?

LEVEL 2 — CHAIN OF RELATIONS

Q6. A is the sister of B. C is the mother of B. D is the father of C. E is the mother of D. How is A related to D?

Q7. Pointing to a lady, Rahul says, "She is the daughter of the woman who is the mother of my mother's husband." How is the lady related to Rahul?

Q8. Showing a photo of a boy, Kavya (a girl) says, "His father is the only son of my father." How is Kavya related to the boy?

Q9. Vikas says, "Aman is the son of the wife of my father's only brother." How is Aman related to Vikas?

Q10. Pointing to a woman, Deepak says, "Her son's wife is the daughter of my father's only son." How is the woman related to Deepak's daughter?

LEVEL 3 — CODED RELATIONS

Code: $P + Q \rightarrow P$ is the **father** of Q | $P - Q \rightarrow P$ is the **wife** of Q | $P \times Q \rightarrow P$ is the **brother** of Q | $P \div Q \rightarrow P$ is the **daughter** of Q

Q11. If $A \div B + C$, how is A related to C?

Q12. If $M - N + O \times P$, how is M related to P?

Q13. Which expression shows that **R is the grandfather of T**?

(a) $R + S + T$ (b) $R \times S + T$ (c) $R - S + T$ (d) $R \div S + T$

LEVEL 4 — FAMILY PUZZLE (Q14 – Q15)

A family has six members — Ravi, Meena, Arjun, Sita, Kabir and Tara.

- Meena has only one child, Arjun.
- Ravi is Meena's husband.
- Meena is Sita's mother-in-law.
- Arjun is Tara's father, and Kabir is Tara's brother.

Q14. How is Kabir related to Ravi?

Q15. How is Ravi related to Sita?

Aur practice chahiye?

→ [IndiaBix: Blood Relation — Introduction & Solved Examples](#)

→ [IndiaBix: Blood Relation Test \(practice questions with explanations\)](#)

4 Tricky Code Analysis (10 Questions)

Dono ke liye. In codes ko dry run karo. Output predict karo, ya agar compilation/runtime error hai (ya program kabhi khatam hi nahi hota) toh reason batao.

Case 1 — Binary Search on an Unsorted Array

```

public class Test1 {
    public static void main(String[] args) {
        int[] arr = {7, 2, 9, 4, 1};
        int target = 4;
        int low = 0, high = arr.length - 1, ans = -1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] == target) { ans = mid; break; }
            else if (arr[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        System.out.println(ans);
    }
}

```

Case 2 — The Mid Overflow Trap

```

public class Test2 {
    public static void main(String[] args) {
        int low = 2_000_000_000;
        int high = 2_100_000_000;
        System.out.println((low + high) / 2);
        System.out.println(low + (high - low) / 2);
    }
}

```

Case 3 — The Stuck Pointer

```

public class Test3 {
    public static void main(String[] args) {
        int[] arr = {1, 3, 5};
        int target = 5;
        int low = 0, high = arr.length - 1;
        while (low < high) {
            int mid = (low + high) / 2;
            if (arr[mid] < target) low = mid;
            else high = mid;
        }
        System.out.println("Found at: " + low);
    }
}

```

Case 4 — Built-in Binary Search Return Values

```
import java.util.Arrays;

public class Test4 {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40};
        System.out.println(Arrays.binarySearch(arr, 40));
        System.out.println(Arrays.binarySearch(arr, 25));
        System.out.println(Arrays.binarySearch(arr, 5));
    }
}
```

Case 5 — Sorting Strings (Capital vs Small)

```
import java.util.Arrays;

public class Test5 {
    public static void main(String[] args) {
        String[] fruits = {"banana", "apple", "Cherry"};
        Arrays.sort(fruits);
        System.out.println(Arrays.toString(fruits));
    }
}
```

Case 6 — Linear Search Without break

```
public class Test6 {
    public static void main(String[] args) {
        int[] arr = {4, 8, 4, 2};
        int idx = -1;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == 4) {
                idx = i;
            }
        }
        System.out.println(idx);
    }
}
```

Case 7 — Recursive Binary Search

```

public class Test7 {
    static int search(int[] a, int lo, int hi, int t) {
        if (lo > hi) return -1;
        int mid = (lo + hi) / 2;
        if (a[mid] == t) return mid;
        if (a[mid] < t) search(a, mid + 1, hi, t);
        else search(a, lo, mid - 1, t);
    }
    public static void main(String[] args) {
        int[] arr = {1, 3, 5, 7};
        System.out.println(search(arr, 0, 3, 7));
    }
}

```

Case 8 — Wrong Starting Value of high

```

public class Test8 {
    public static void main(String[] args) {
        int[] arr = {1, 3, 5};
        int target = 9;
        int low = 0, high = arr.length;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] == target) {
                System.out.println("Found");
                break;
            } else if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }
}

```

Case 9 — Comparing Wrapper Objects

```

public class Test9 {
    public static void main(String[] args) {
        Integer a = 127, b = 127;
        Integer c = 1000, d = 1000;
        System.out.println(a == b);
        System.out.println(c == d);
        System.out.println(c.equals(d));
    }
}

```

Case 10 — Searching in an ArrayList

```
import java.util.ArrayList;
import java.util.Arrays;

public class Test10 {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>(Arrays.asList(5, 10, 15, 10));
        System.out.println(list.indexOf(10));
        System.out.println(list.lastIndexOf(10));
        System.out.println(list.indexOf(99));
        list.remove(Integer.valueOf(10));
        System.out.println(list.indexOf(10));
    }
}
```

5 Code Modification Challenge

Neeche ek basic **Linear Search** ka code diya gaya hai jo sorted array mein target dhundta hai:

Searcher.java — Starting Point

```
public class Searcher {
    public static void main(String[] args) {
        int[] arr = {2, 4, 4, 4, 4, 9, 12, 15, 18, 21};
        int target = 18;
        int index = -1;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                index = i;
                break;
            }
        }
        System.out.println("Index: " + index);
    }
}
```

Tumhara Task — Is code ko modify karo taaki yeh:

1. Linear Search ki jagah **Binary Search (iterative)** use kare — $mid = low + (high - low) / 2$.
2. Dono approaches mein loop **kitni baar chala (steps)**, count karke print kare — target = 18 ke liye Linear vs Binary compare karo.
3. Duplicate hone par (target = 4) **first occurrence** ka index return kare — koi bhi random 4 nahi. *(Hint: match milne par answer save karo aur $high = mid - 1$.)*
4. Target na mile (target = 10) toh "Not Found" print kare, aur saath mein batao ki use **kis index par insert** karein taaki array sorted rahe.

6 Submission & Deployment

Dono students ko apna code `task_10/` folder mein push karke GitHub link submit karni hai.

```
task_10/
├── Akshara/
│   ├── SumOfDigits.java
│   ├── ReverseTheNumber.java
│   ├── ArraySearch.java
│   ├── CheckSorted.java
│   ├── EvenDigitNumbers.java
│   ├── BinarySearchBasic.java
│   └── BloodRelations.md      (15 answers + har ek ke steps / family tree)
├── Hardik/
│   ├── BinarySearch.java
│   ├── SearchInsertPosition.java
│   ├── FirstBadVersion.java
│   ├── SqrtX.java
│   └── FirstAndLastPosition.java
├── TrickyCodeAnalysis.md      (10 cases ke answers + reasoning)
└── Searcher.java              (modified – binary + step count + first occurrence + insert posit
```